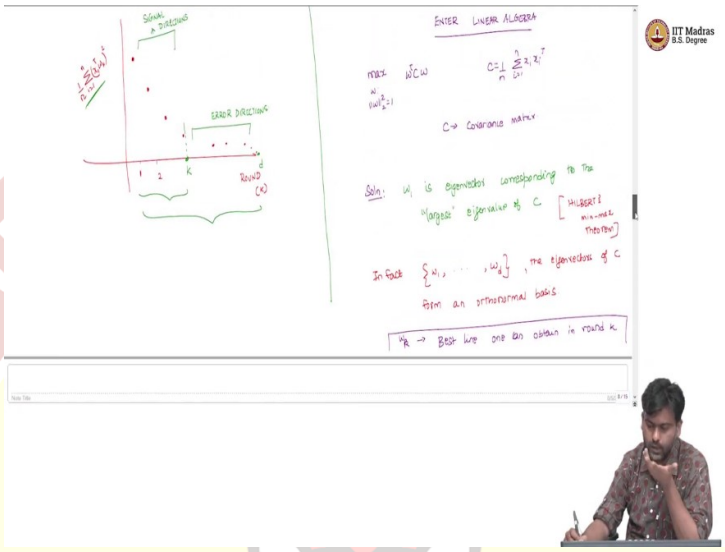
IIT Madras

ONLINE DEGREE

Machine Learning Techniques
Professor Arun Raj Kumar
Department of Computer Science and Engineering
Indian Institute of Technology Madras
Principal Component Analysis: Part 2

(Refer Slide Time: 00:14)



So this is one way to understand this algorithm. Here is another way. So we will look at some linear algebra. So let us say, enter linear algebra now, for those who are familiar with linear algebra, we are already seeing these connections. Nevertheless, I will try to explain it now. So, now, we did talk about this briefly last time, but let us make this more precise now.

So, this is the problem that we are trying to solve, where C is $\frac{1}{n}\sum_{i=1}^{n}(x_i x_i^T)$ and this C is called the covariance matrix with some caveats with covariance matrix in general, and when you have a matrix like this, the solution to this problem is nothing but the Eigen vector corresponding to the largest Eigen value of this matrix.

So, the solution to this problem turns out to be w_1 which is the Eigen vector corresponding to the largest eigenvalue of C. So, this comes from a general theorem, which is called the Hilbert min-max theorem. I am sure if you had taken a linear algebra course, the foundational course so, you would have encountered this at some point a variant of this theorem.

So, this is from that. In fact, more is known. So, Hilbert theorem says more. So, in fact, not just w_1 you can go ahead and find out w_1 to w_d. So, the Eigen vectors of C form orthonormal basis. So, the orthonormal basis is of interest for us which does the error minimization of

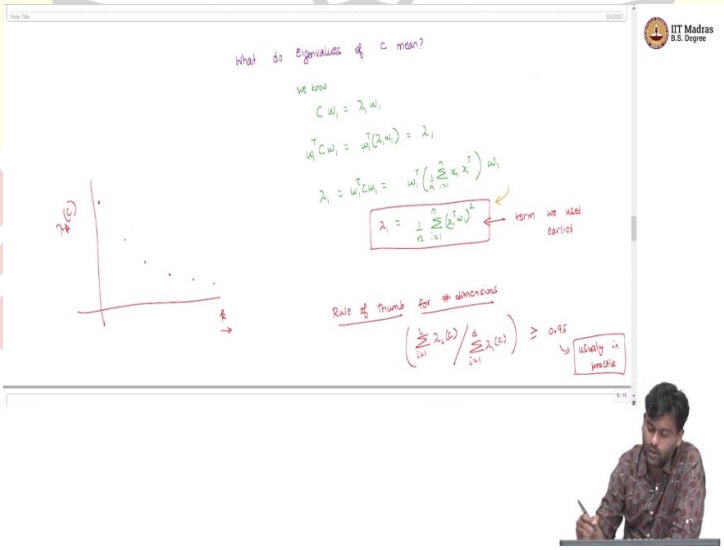
centred data, happens to be just the Eigen vector basis of the covariance matrix. That is essentially the takeaway from the linear algebra point of view.

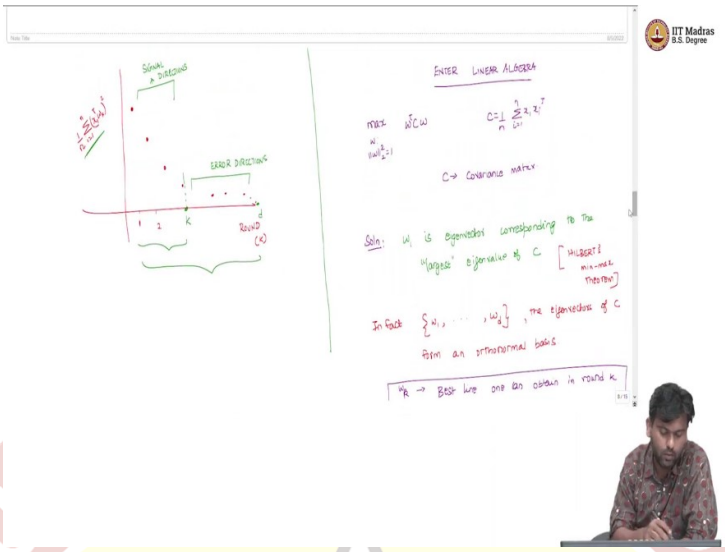
Now, this w_k, the kth Eigen vector or the Eigen vector corresponding to the kth largest Eigen value is exactly the best line one obtains after round k, in round k rather. So, the previous procedure that we said, where we go one step at a time to get these w's, in the kth round, you will get w_k and that w_k is exactly same as the w_k that you would get if you did this.

So, this is again a conclusion that you can derive from Hilbert min-max theorem, that if you try to maximize this same quantity, well, you want w to be norm 1, but then w should be orthogonal to an already existing subspace, well, then, that subspace is formed with the top k-1 Eigen vectors, then the w_k would be the kth Eigen vector.

That is also Hilbert min-max theorem, but we can either understand it that way or we can simply say that this is the line that you get in the round k. Remember, the line that we got in round k will by definition is orthogonal to the previous lines, because the data set that we used were all orthogonal to the to the eigenvectors, to the lines that we derived in the previous rounds by construction.

(Refer Slide Time: 04:09)





So, this is fine. So, we can think of this as eigenvectors and eigenvalues of the covariance matrix, but what does it even mean. So, what do the Eigen values mean? What do Eigen values of C mean? Can we say, can we interpret this slightly differently? Or we should just be happy to say that these are Eigen values of C so that that is fine, but then that does not really tell us anything more than the fact that it is a known, well studied object.

So, can we say something more? Can we think of this in a more geometric point of view? Well, let us try. So, we know $Cw_1 = \lambda_1 w_1$, where Cw_1 is the Eigen vector and λ_1 is, let us say the largest Eigen value. So, corresponding to Eigen vector $w_1$or in this, this works for you in general w_k, where λ_1 will be replaced by λ_k. But for now let us start with w_1. Now what${w_1}^T C w_1$?

Well, that would be${w_1}^T \lambda_1 w_1$, that would simply be λ_1. Because ${w_1}^T w_1$ is one by our definition of looking for lines through unit length. So λ_1=${w_1}^T C w_1$, which equals${w_1}^T \left(\frac{1}{n}\sum_{i=1}^{n}({x_i}^T x_i)\right) w_1$

, that is your C times w_1, which is $\frac{1}{n}\sum_{i=1}^{n}({x_i}^T w_1)^2$. So, now, this term must be, this is λ_1 and this term must be familiar.

So, this is exactly the term we used earlier. So, when did we use it when we used it to find out how many rounds we should run this algorithm for? So, this is basically this term is exactly the Eigen value the highest Eigen value. So, now, basically then we can do the following I

mean whatever we did last time, I mean before can also be interpreted like this. So, you have k which is the number of rounds, or the kth Eigen value and small k.

And now you have $\lambda_k$of the covariance matrix. And if you plot that, that those guys are going to fall down. So, our λ_{ik} of C is the kth largest Eigen value of C because this is exactly the, you know the term that we used earlier too. So now the rule of thumb, so, rule of thumb for how many dimensions, for number of dimensionslast time, like how we just said like a few minutes back would be to look at the following quantities $\frac{\sum_{i=1}^{l}\lambda_i(c)}{\sum_{i=1}^{d}\lambda_i(c)}$, if this ratio is greater than or equal to 0.95. Usually, in practice, this is a thumb rule, usually in practice, then we are saying that while we are retaining some 95% of the information in this dataset, so, that is the general idea.

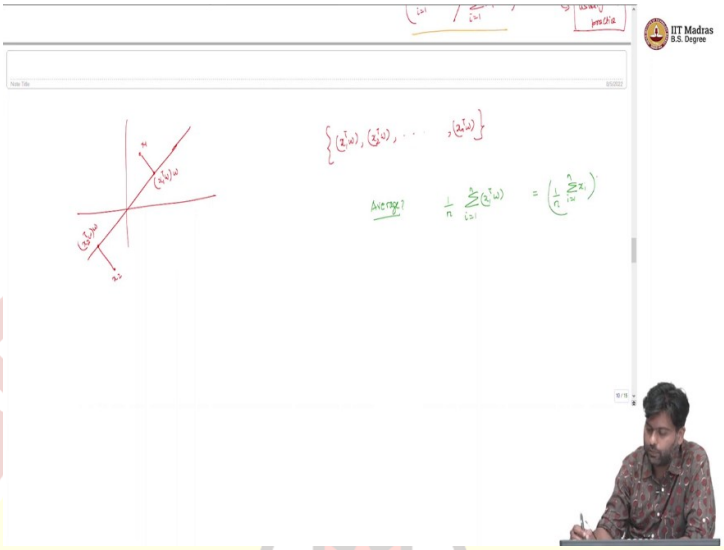
Now, for this quantity to make sense, so, summing up the top k, top l and then dividing it by the entire thing to be greater than 0.95, all of these have to be, positive values, or at least non negative values,? But then that is true, because of the fact that you know, your $\lambda_1$in general, λ_k will be $\frac{1}{n}\sum_{i=1}^{n}(x_i{}^T w_k)^2$e. So this will be just the average of a bunch of squared terms. So all of this will be positive.

So in fact, this covariance matrix, this kind of tells us that the covariance matrix has non negative Eigen values, which means the covariance matrix is for people who have seen this is a positive semi definite matrix, So this is also a proof of that. Anyway, so the main point is that this quantity makes sense. So because these are positive numbers, so you can look at the ratio and see when it crosses 0.95, and then use that as a as a value that you want.

So what we are basically doing is that we are saying that we are maximizing the quantity that we are maximizing is actually the Eigen value of the dataset that we saw of the covariance matrix of the dataset. But that still does not tell us what this quantity is. So it just explains us the quantity equation in terms of Eigen values.

So but that does not really tell us what is this quantity about? So can we understand this quantity in a more simpler way? Specifically, this quantity? I mean, of course, this is, it happens to be the Eigen value of the covariance matrix. That is well and good. But what is it? So can we say anything about this term?

(Refer Slide Time: 09:26)

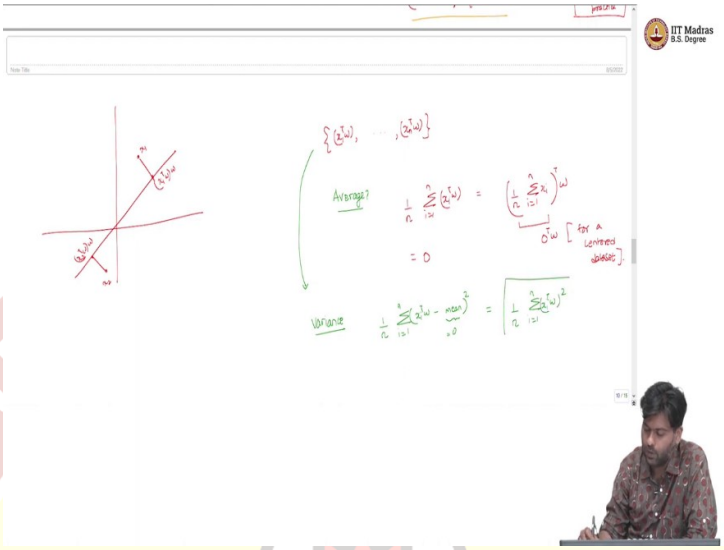


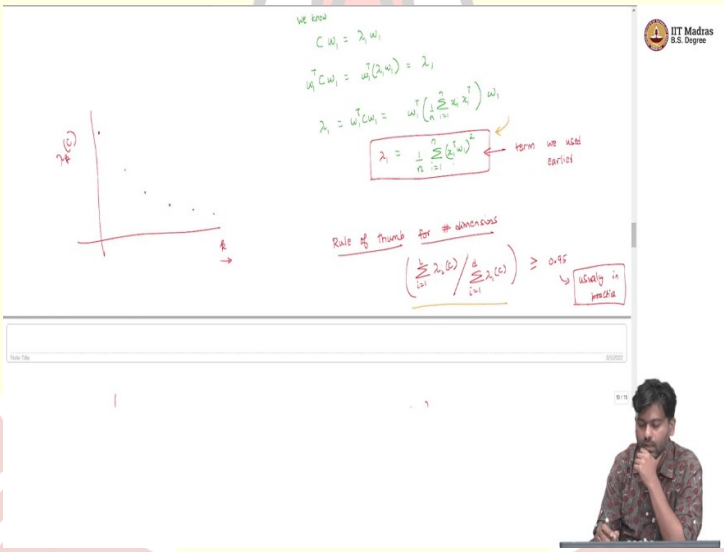
So now, let us say we take, again, a bunch of data points on the line. So maybe there is an x_1, and then this point becomes$(x_1{}^T w)w$, so on, maybe there is an $x_2$becomes $(x_2{}^T w)w$, and so on. Now, let us say we fix w we collect these values $(x_1{}^T w)$, $(x_2{}^T w)$……$(x_n{}^T w)$.

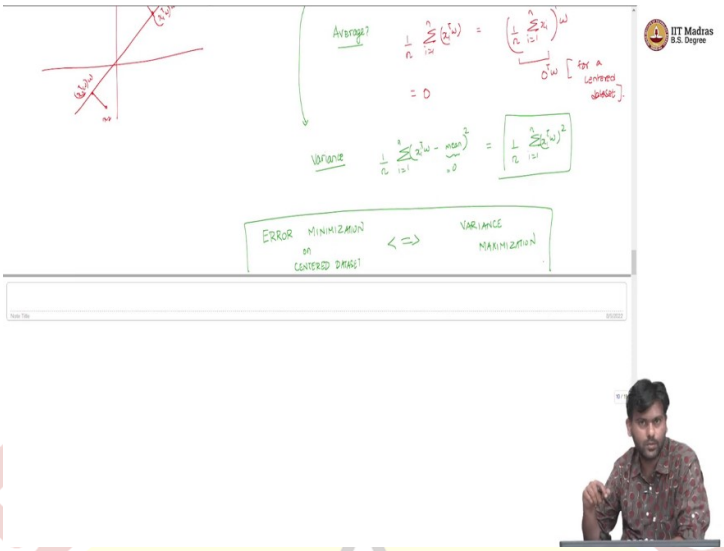
Let us say we collect these values. So these are like the, in some sense the, if you square this, they will be the length squared of the projection, the proxy for each of these data points on the line w. So I am not saying the line w is the best line or anything, it could be any line. And then we are collecting this bunch of data points.

Now, the first question is, what is the average of these values? Average? What would this be $\frac{1}{n}\sum_{i=1}^{n}(x_i{}^T w)$. I am not saying anything about w being the best line or anything, pick some arbitrary line and then project onto that line, compute these $(x^T w)$values and then compute their averages. So what is this value going to be? So you may want to pause and think about this before I tell you the answer now.

(Refer Slide Time: 11:00)







So this value is just $\frac{1}{n}\sum_{i=1}^{n}(x_i{}^T w)$. Now this value for a centre data set, this value is just the mean, but then the mean is 0 for a centred data set. So, we are assuming that the data set is centred. So this is a centred data set. Now, what does that mean? That means that the average is actually 0 for a centre data set. So this is it is a number, it is 0. So now, what can we say about the same set of values?

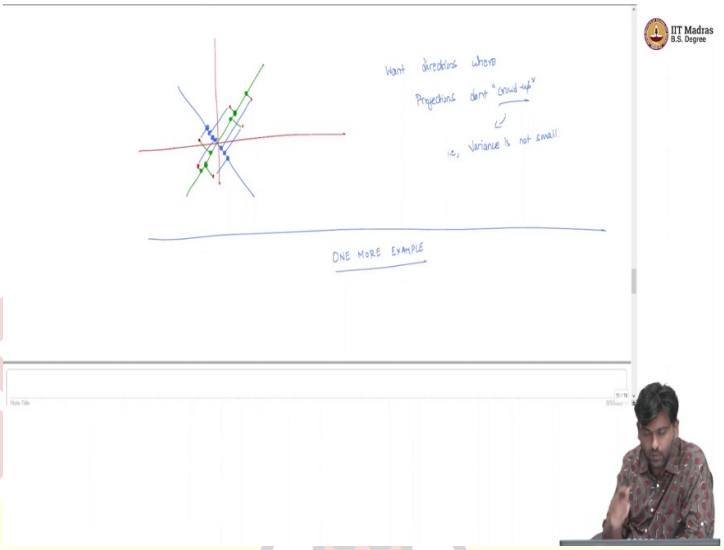
But then let us look at the variance. So, how much spread? Is there in this bunch of values? Now, how would we compute the variance? Well, we can do the $\frac{1}{n}\sum_{i=1}^{n}(x_i{}^T w - mean)^2$, where the mean or the average is the average of this bunch of numbers. But we know that the mean is 0.

That is what we just looked at. So, which means this value is just going to be $\frac{1}{n}\sum_{i=1}^{n}(x_i{}^T w)^2$. So what is this telling us, this is telling us that you pick an arbitrary w and then project all your centre data points on to this w and then just compute the variance of the projected values. Now that variance is exactly the term that we were using earlier. So it is exactly this term that we are trying to maximize.

And it is exactly the term which is equal to the Eigen value. So now, this is interesting. So now at least we know, you know, what are we doing when we are doing this maximization. So basically, then we can conclude the following. So, error minimization, which is what we started this algorithm with, on centred data, on the centering is important because only then the equivalence that we are putting out now holds is equivalent to variance maximization.

So if you find the line where the error is minimum, then it also means that it is the same line where the variance of these projected values is as high as possible.

(Refer Slide Time: 14:02)



So to see why this has to happen geometrically. Of course, algebraically. We have seen this but then geometrically, also, you can kind of convince yourself that maybe you have a bunch of points, like this. And maybe the green line is the best line, maybe there is also this, maybe there is also this blue line. Now you can project it onto either of these lines. If we projected onto the green line. I am going to get the proxies as follows.

The green dots are my proxies. Whereas if I project it onto the blue line, the blue dots are the proxies with the red points. Now, if you focus on the green points, you can see that they are kind of spread out then the blue points which are clouding, So now the problem is if data points all crowded up, in fact, if they go to the same point, then there is no way you can distinguish this one data point from the other.

Whereas, if they are spread out, then this distinguishing capability is still present in this data points. You do want compression, but you also want to be able to distinguish one data point from another because in downstream, you are going to use this compressed representation for some other task, let us say supervised learning, then you will still want to say one data point from the another.

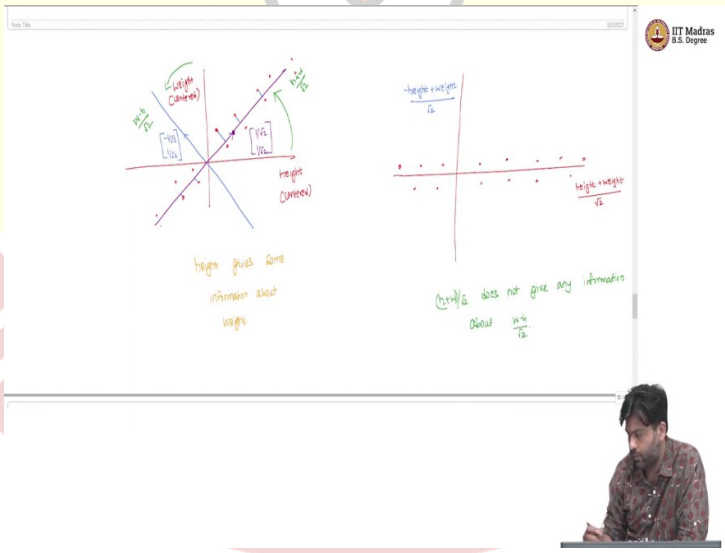
So the distinguishing capability is still necessary, which is lost if you project it along a wrong line. So the wrong line where all the data points crowd closer to each other, which means if the variants of the projector data points is small, then that is a problem. In fact, if, if all data

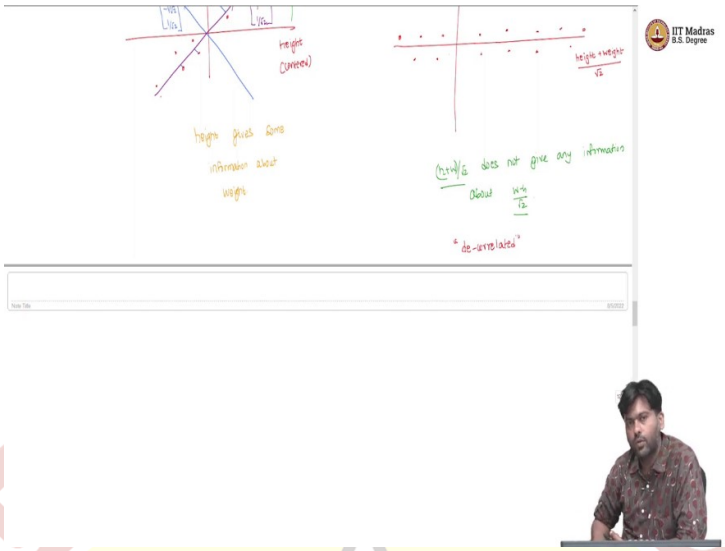
points actually fell on the same line, now, if you look at the perpendicular line, and then think of that line as your actual line, then what would happen is all the data points would reject onto the 0 vector, which means that there is no way to distinguish one data point from another. And that is a bad thing. So because you cannot really use this representation to do much later. So what you really want to do is you want to find directions where the projections do not crowd.

So we want directions where projections do not crowd. So I am using this crowd up in a loose fashion. But for us, the definition of crowd up is that variance implies that this variance is not small. So we want variance to be as high as possible. And in that case, we would have found a line where the spread is as high as possible.

In other words, the information retained is as high as possible. So that is one other way to think about PCA itself. So this algorithm itself, of course so this is one way to think about it. And there is one more example that we will see, and then we will conclude this initial algorithm that we are starting to look at, probably use it this.

(Refer Slide Time: 17:33)





Here is one way to understand what is happening. Let us say we are given, again, a bunch of data points centred, and so on. And let us say this is the height, and this is the weight, they are centred. So that is why you see negative values. So let me put it centred, centred, and so on. Now, if the main thing that we are observing is that height gives some information about weight, so if I tell you what the height is, if the height increases, the weight also tends to increase.

So that is the main observation here. And that is why we started with fitting lines and so on. But the more importantly, height and weight are not completely, loosely saying, independent of each other. So I should be careful when I use the word independent. But let us say intuitively, height and weight are not independent of each other, because height gives me some information about the weight.

So if I choose to represent these data points as height and weight as 2 numbers, height and weight, then 1 number gives me some information about the other number. Now, if I did this algorithm, where do we find these lines? So let us say we found this line, which is the best line and this line, $\boldsymbol{w}$ is going to be [1 1].

Well, of course, it cannot be [1 1], because it has to be of length 1, let us say I normalize this and the length is $\left[\frac{1}{\sqrt{2}}\frac{1}{\sqrt{2}}\right]$, then it means that the first so if I am allowed to use only one number to represent each data point, then that number would be the dot product of each data point along this direction. Well, the data point itself is just height and weight.

Now this direction is $\left[\frac{1}{\sqrt{2}}\frac{1}{\sqrt{2}}\right]$, which means that what I would actually be storing is the value height x $\frac{1}{\sqrt{2}}$+ weight x $\frac{1}{\sqrt{2}}$ which is (height+weight)/$\sqrt{2}$. Now, for whatever reasons, if I went ahead and found the second line, so which is, which would fit the error vectors, it would be along the perpendicular direction,we know that and that vector would simply be $\left[-\frac{1}{\sqrt{2}} \quad -\frac{1}{\sqrt{2}}\right]$. Now that axis, so that value projection along that value would be (-height+weight)/$\sqrt{2}$. Also, if I think of that vectors, you know, $\frac{1}{\sqrt{2}}$, -$\frac{1}{\sqrt{2}}$does not really matter. So it can, it can be either.

So if I, now thought of plotting the same data set, but then by thinking of these 2 axis values, so instead of height and weight, I am going to compute 2 numbers (height + weight)/ $\sqrt{2}$, and (weight- height)/ $\sqrt{2}$, and then plot the same data set, how do you think that dataset would look like? This is a good place to pause and think, let me tell you how the data set would look like. The data set would look something like this.

So basically, what has happened is, you can think of it as if this high axis gotten transformed, rotated, in this case by 45 degrees to get this(height + weight)/ $\sqrt{2}$, because there is a scaling happening, which I am not indicating here. But yes, you can think you can imagine that it is on the weight axis has gotten rotated this way to become (weight- height)/ $\sqrt{2}$. Now, if I think of looking at the same data set in this direction, well, that would exactly look like this. If we plot now what is the point here.

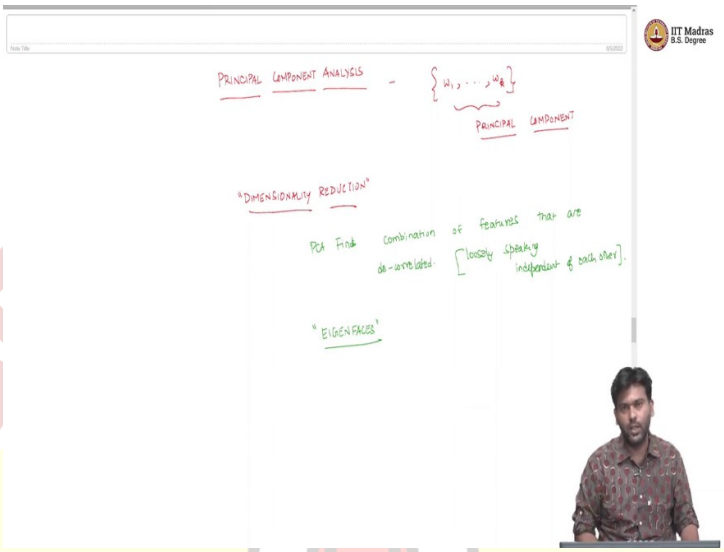
So, what is the point of plotting this well, this is telling us that well, now if I told you the x axis value, which is (height + weight)/ $\sqrt{2}$ value, there is nothing that you can really say about height (weight- height)/ $\sqrt{2}$, if (height + weight)/ $\sqrt{2}$ increases along this direction, there is nothing really happening for (weight- height)/ $\sqrt{2}$, it can either be above the axis or below the axis, but then there is no real pattern there.

Whereas in the previous case, as height increase the weight tend to increase and height decrease the weight also tend to decrease here nothing happened, like that is happening. So in other words, (height + weight)/ $\sqrt{2}$ does not give any information about (weight- height)/ $\sqrt{2}$. So well, in other words, these two directions, so the w_1 and w_2 directions, and then the values that we project on to these directions.

So the right, or the terminology is that these directions are de-correlate for this data set. So the directions where every direction gives you some new information that the previous

directions are not giving you, you are trying to find those directions. So that is what our algorithm is essentially doing.

(Refer Slide Time: 23:06)



And this algorithm well is called the principal component algorithm. You might have come across this algorithm already, if you are taken MLF course. Nevertheless, we thought it would be good to start with this algorithm, because some of the ideas that we will see now, which you may not have seen, the MLF of course, would lead us to something more fundamental in machine learning.

And the w_1 to w_k that we have managed to find these vectors are called as the principalcomponents. And what our example, now say is that these principal directions are in fact, you know, de-correlated. So, the projections along these directions kind of for de-correlated.

So, essentially, the algorithm is given the data set, find the covariance matrix, find the top k Eigen vector and Eigen values, project your data set along the Eigen vector directions, and then that would give you compressed representation, because we are going from R^d to R^k, where k might be much, much smaller than d. This is also a dimensionality reduction technique.

So, our essential dimension that we believe for this data set is only k, It is the Top k Eigen vectors which constitute to 95% of the variance. It can explain 95% of the variance in our dataset. That is why the 0.95 that we used, where it said its information is essentially the

amount of variance explained by the top k Eigen vectors. So, so we should think of each of these directions as a dimension, essentially.

And what PCA does, essentially is now finding new axis, so finding PCA finds a combination of features. So by when I say $x_i^T w_1$, it means that you are weighing these features in x_i using the weights given by these Eigen vectors. So it finds a combination of features that are de-correlated loosely speaking, independent of each other.

So, this is a very fundamental algorithm in data science in dimensionality reduction and so on. And one of the basic algorithms that we will start with for representation learning. So, next time what we will see is one simple example where this algorithm can be applied. And this is this example is called as the Eigen faces example.

So, a face recognition type of a system that you can build using the principal component analysis algorithm will demonstrate that, at least show you some pictures. That is the first thing. And then what we will do is we will identify some potential issues with PCA, and places where it may not necessarily work that well.

And once we have identified those, we will try to fix them and as a matter of fixing them, that will reveal to us some interesting insights about how to look at a dataset, what is important in a data set and so on, which will lead us to more foundational ideas in machine learning. So, all that will come in the next few lectures. For now, we have seen PCA and I hope you are able to appreciate the foundational fundamental nature of this idea and this algorithm. Thank you.